# Break It Down —项目技术报告

在定制时代，让每个人都能定制自己的产品。

想象一下，也许你是一个渴望拥有绝妙产品灵感的白日梦想家；或许你是一个追爆款但又想差异化竞争的 TikTok 电商卖家；或许你恰好紧缺一批特殊零部件（机械比赛/具身智能初创/…）。这时，你希望能从零件开始自己手搓。

但现实给了你当头一棒。当你把一张手绘草图或一张实物照片发给 1688 的工厂老板时，对方只会连环追问：”你的零件需要用什么材料？总单量要做多少？你的工艺要求和公差是多少？”

在这个号称”万物皆可造”的时代，曾经绝大部分的定制需求，死在了”专业性”的门槛上。消费者有想象力，工厂有生产力，但他们之间缺少一种”让一切人有能力创造一切”的共同语言。

Break It Down 就是这种语言。

---

## 一、产品定位

**一句话定位：**
Break It Down 是一个 AI 驱动的物品拆解与供应链分析平台——用户输入任意物品（拍照或文字），系统自动将其逐层拆解至最基础原材料，并可对任意零件发起完整的生产可行性分析。

**解决什么问题：**
传统方式需要大量行业经验和人工检索，Break It Down 将这一过程压缩为：**上传图片 → 点击拆解 → 获得从成品到原矿的完整拆解图谱，以及一份可落地的生产分析报告**。

---

# 二、亮点功能（核心能力与差异点）

## 2.1 多模态物品识别

用户可通过两种方式输入：

- **图片识别**：上传照片，调用阿里云通义千问视觉模型（`qwen-vl-plus`）识别物品名称、类别、简要描述，并自动生成 Wikimedia 图片搜索词，为节点配图。
- **文字识别**：直接输入物品名称，调用文本模型完成结构化识别，返回标准化的 `IdentificationResponse`(name / category / brief_description / icon / searchTerm)。

## 2.2 递归拆解树（最深 6 层）

识别完成后，用户点击任意节点即可触发该节点的 AI 拆解，系统返回其直接子组件列表，并在 React Flow 画布上实时渲染为可交互的有向无环图。

拆解规则严格约束：

- 最大深度 6 层，终止节点必须是预定义的基础原材料（铁矿石、铜矿石、硅砂、原油、煤炭、锂、木材、棉纤维、天然橡胶等 12 种）
- 支持三档细致度（极简 / 中等 / 详细），由用户在 Setup 页面通过滑块调节
- 支持幽默度和专业度两个维度的 Prompt 风格调节
- 每个节点携带 emoji 图标 + Wikimedia 配图，视觉辨识度极高

## 2.3 组件详情面板（Component Detail Panel）

点击任意已拆解节点，可一键生成该组件的制造工艺详情面板，内容包括：原材料来源、主要加工工序、行业应用场景等，由 AI 实时生成并缓存至本地 Session，避免重复调用。

## 2.4 七步多 Agent 生产分析（核心差异点）

这是 Break It Down 最具技术深度的功能。用户在拆解树中选择任意零件，进入”生产模式”，系统启动 **7 个专业 Agent 串行协作**，完成从零件到上市的完整分析链路：

每个 Agent 均有独立的 System Prompt，通过 DeepSeek API 的 **SSE 流式输出**实时推送到前端，用户可以看到 AI 逐字生成内容，体验感极强。

| 步骤 | Agent 角色 | 输出内容 |
|---|---|---|
| Step 1 | 产品规划专家 | 目标用途、价格区间、推荐材料、产品特性 |
| Step 2 | 竞品分析专家 | 市场价格带、主要竞品、定价建议、市场趋势 |
| Step 3 | 供应商检索专家 | 3-5 个供应商选项（名称/规格/价格/可靠性/交期） |
| Step 4 | 定制顾问 | 3-5 个定制化问题，引导用户明确需求参数 |
| Step 5 | 制造工艺专家 | 3-4 种工艺方案（成本/风险/碳排放） |
| Step 6 | 成本 + 风险 + 碳排放三路并行评估 | 成本分解、风险矩阵、碳足迹评级（A-E） |
| Step 7 | 综合决策专家 + 最终报告生成器 | break/keep 建议 + 四章节完整商业报告 |

## 2.5 模板拦截与内存索引

`lib/template-interceptor.ts` 实现了一套轻量级的两级匹配引擎：

- **Level 1 精确匹配**：用户输入包含关键词（或关键词包含用户输入），置信度 1.0
- **Level 2 模糊匹配**：Levenshtein 编辑距离 $\leq 2$ 且长度比 $\geq 0.6$，置信度 0.8

内存索引每 5 分钟重建一次(TTL 机制),避免每次请求全表扫描,支持 `invalidateIndex()` 强制刷新。

## 2.6 会话持久化与历史回溯

所有拆解会话自动保存至 SQLite 数据库，包含：完整树结构 JSON、节点位置、识别结果、Prompt 设置、分析缓存、生产分析进度（每个零件独立存储，支持断点续分析）。历史页面支持一键重新加载，恢复完整画布状态。

## 2.7 沉浸式首页视差动画

首页采用 Framer Motion 实现多层视差滚动：背景视频模糊淡出 → 等轴测立方体三面拆解动画 → Logo 放大虚化消失 → 品牌名称渐入 → 宣传语出现 → 滚动进入产品介绍区。整个动画由 `scrollYProgress`（0-1）驱动，无任何 JS 定时器，性能极佳。

# 三、创意来源

## 3.1 洞察：降维打击千亿级定制市场

目前，定制市场（C2M）的痛点不在于工厂产能不足，而在于”前置沟通成本极高”。

Break It Down 的市场策略是”零门槛拦截”——让过去只有专业硬件工程师才能做的事情，被每一个普通人、独立设计师、创客和中小电商卖家完成。我们抢占了巨大定制市场的潜力群体：那些因为无法描述数据要求而无法个性化定制的海量用户。

当门槛降为零时，潜在需求就会呈指数级爆发。我们搭建的是”全民定制时代”的基础设施。

想象一位工业设计系学生在路边咖啡馆看到一把设计奇特的椅子。他没有去搜品牌，而是打开 Break It Down 拍了张照片——AI 秒级将椅子拆解，从蒙皮材质、内部填充海绵到极简的金属骨架，层层展开。他惊讶地发现，那看似昂贵的航天铝支架，在 1688 上的基础型材成本竟然不到 50 元。这种”看穿世界真相”的快感极具成瘾性。

## 3.2 极致粘性：为什么用户用了第一次，就会无限次使用

- **造物者的”随身计算器”**：用户在生活中看到任何有趣的、昂贵的物件，都会忍不住拍照上传——”这个东西如果我自己做，成本到底是多少？”我们满足了人类底层的好奇心与掌控欲。
- **个人数字资产库**：每一次拆解、每一次调整材质，都保存在用户的专属库中。当灵感积累在平台上，用户就再也离不开了。
- **无缝嵌入巨头工作流**：Break It Down 可作为淘宝、1688、TikTok、小红书等超级生态的”转化引擎”。中小电商卖家刷到爆火创意产品，通过浏览器插件一键拆解，系统瞬间生成基础 BOM 表和打样成本——将”看爆款”和”造爆款”之间的距离从几周缩短到几秒。
- **双边网络效应**：对供应商而言，用户带着 Break It Down 生成的结构化数据来询价，供应商收到的不是”我想做个圆圆的东西”，而是符合工业标准的 BOM 表，可直接导入 ERP 秒级回传报价。曾经需要 15 天的沟通，缩短至 15 分钟。供应商甚至会主动要求客户：”请先用 Break It Down 跑一遍数据再发给我。”

## 3.3 多 Agent 架构的灵感来源

生产分析的 7 步 Agent 设计，来源于对真实制造业决策流程的抽象：一个新产品从概念到量产，真实需要经历产品规划 → 市场调研 → 供应商寻源 → 定制需求确认 → 工

艺选型 → 成本/风险/环保评估 → 综合决策这几个阶段，每个阶段都有专业的人在负责。

我们将这个流程”数字孪生”化，每个 Agent 对应一个真实岗位角色，拥有独立的专业 System Prompt，Agent 之间通过结构化 JSON 传递上下文，形成一条完整的”数字供应链顾问”流水线。

### 3.4 路演场景的工程化思考

在实际演示中，等待 AI 实时生成拆解结果会造成明显的卡顿感，影响展示效果。因此我们设计了”模板预热”机制：提前为 32 个精选品类（覆盖大国重器、极客定制、生活黑科技、绿色新消费四大类）生成完整的拆解树和 7 份 Agent 报告，存入数据库，演示时毫秒级响应，同时保留真实 AI 路径供非模板物品使用。这是工程实用主义与产品体验的平衡。

---

## 四、技术架构

### 4.1 整体架构图

```
                    用户浏览器
Next.js 15 App Router  ·  React 19  ·  Framer Motion
React Flow（画布）  ·  Tailwind CSS  ·  NextAuth Session

                    HTTP / SSE

              Next.js API Routes (Node.js)

/api/identify          → 图片识别 (Qwen Vision)
/api/identify-text     → 文字识别 + 模板拦截
/api/deconstruct       → 递归拆解 (Qwen/Custom AI)
/api/knowledge-card    → 组件详情面板生成
/api/plugins/smart-manufacturing        → 7-Agent 批处理
/api/plugins/smart-manufacturing/stream → 7-Agent SSE流式
/api/sessions          → 会话 CRUD
/api/templates         → 模板读取
```

```
/api/wikimedia-search  → 图片搜索
```

```
AI 服务层                          数据持久层

通义千问                           Prisma 6 ORM
qwen-vl-plus                       SQLite (dev)
(视觉固定)
                                   User
通义千问                           DeconstructionSession
qwen-plus                          ProductionAnalysis
(文本备选)                           Progress
                                   TemplateSession
自定义 AI                          (32 预设模板)
OpenAI兼容
(文本优先)

DeepSeek
deepseek-
chat
(生产分析)
```

## 4.2 AI 模型分工

| 功能模块 | 使用模型 | 原因 |
|---|---|---|
| 图片识别 | 阿里云 `qwen-vl-plus` | 固定使用，多模态视觉能力强，中文理解好 |
| 文字识别 | 自定义 OpenAI 兼容接口或 `qwen-plus` | 优先使用用户配置的自定义接口 |
| 递归拆解 | 自定义 OpenAI 兼容接口或 `qwen-plus` | 同上，支持灵活替换 |
| 组件详情面板 | 自定义 OpenAI 兼容接口或 `qwen-plus` | 同上 |
| 生产分析 7-Agent | `deepseek-chat` | DeepSeek 在结构化 JSON 输出和推理链方面表现稳定，成本低 |

环境变量配置逻辑（lib/ai-client.ts）：

```
if (AI_BASE_URL && AI_API_KEY) // 使用自定义 OpenAI 兼容接口（文本）
else                           // 使用阿里云 DASHSCOPE_API_KEY（qwen
    -plus）
// 视觉识别始终固定使用 qwen-vl-plus，不受上述配置影响
```

## 4.3 多 Agent 协同架构（重点）

这是本项目最核心的技术亮点，值得深入展开。

### 4.3.1 架构设计

生产分析模块实现了两套 API 端点：

- `POST /api/plugins/smart-manufacturing` —**批处理版本**，Agent 串行执行，适合后台任务
- `POST /api/plugins/smart-manufacturing/stream` —**SSE 流式版本**，每个 Agent 实时推送输出，前端逐字显示

每个 Agent 是一个独立的”角色单元”，拥有：

1. 专属 System Prompt（定义角色、职责、输出格式）
2. 结构化的 User Message 构建函数（注入上下文数据）
3. 标准化的 JSON 输出 Schema（供下游 Agent 消费）

共 10 个 Agent，角色与输出 Schema 如下：

### 4.3.2 Agent 间的数据流与上下文传递

每个 Agent 的 `buildUserMessage` 函数接收一个统一的上下文对象，按需注入字段：

```
const {
  partName,            // 零件名称（所有 Agent 共用）
  partSpecs,           // 零件规格
  productPlan,         // 产品规划结果（竞品分析、供应商、最终报告注
    入）
  competitorAnalysis,  // 竞品分析结果（最终报告注入）
```

| Agent | 角色 | 输出 Schema 关键字段 |
|---|---|---|
| 产品规划专家 | 新产品规划 | `productPlan`: useCases, targetPriceRange, materials, features |
| 竞品分析专家 | 市场竞品分析 | `marketResearch`: marketPrice, competitors[], pricingAdvice, marketTrends |
| 供应商检索专家 | 供应商寻源 | `suppliers[]`: name, specs, price, reliability(1-10), leadTime |
| 定制顾问 | 需求参数确认 | `questions[]`: question, type(text/number/select), options |
| 制造工艺专家 | 工艺方案生成 | `processes[]`: name, description, cost, risk, carbonEmission |
| 成本分析师 | 成本拆解 | `totalCost`, `breakdown`: material/processing/shipping/other |
| 风险评估专家 | 工程风险评估 | `riskLevel`(低/中/高), `risks[]`, overallAssessment |
| 碳排放评估专家 | 碳足迹评级 | `totalEmission`, `breakdown`, `rating`(A-E), analysis |
| 综合决策专家 | 生产决策 | `recommendation`(break/keep), confidence(0-100), reasoning, keyFactors |
| 最终报告生成器 | 商业策略报告 | businessBase, supplyChain, productionTimeline, lifecycle |

```
  selectedSupplier,     // 用户选定的供应商（最终报告注入）
  selectedProcess,      // 用户选定的工艺方案（成本/风险/碳排放注入）
  customizedParams,     // 用户回答的定制参数（工艺 Agent 注入）
  costData,             // 成本分析结果（综合决策注入）
  riskData,             // 风险评估结果（综合决策注入）
  carbonData,           // 碳排放结果（综合决策注入）
  analysisResult,       // 三路评估汇总（最终报告注入）
  selectedMaterials,    // 选定材料（供应商 Agent 注入）
  selectedBudget,       // 预算（供应商 Agent 注入）
  selectedFeatures,     // 产品特性（供应商 Agent 注入）
} = data;
```

完整数据流如下：

```
用户选择零件（partName）


[Agent 1: 产品规划专家]
  输入: partName
```

```
  输出: { useCases, targetPriceRange, materials, features }


[Agent 2: 竞品分析专家]
  输入: partName + Agent1.productPlan
  输出: { marketPrice, competitors, pricingAdvice, marketTrends }


[Agent 3: 供应商检索专家]
  输入: partName + selectedMaterials + selectedBudget + selectedFeatures
  输出: { suppliers: [{ name, specs, price, reliability, leadTime }] }

          用户选择供应商

[Agent 4: 定制顾问]
  输入: partName
  输出: { questions: [{ question, type, options }] }

          用户回答定制问题

[Agent 5: 制造工艺专家]
  输入: partName + customizedParams
  输出: { processes: [{ name, description, cost, risk, carbonEmission }] }

          用户选择工艺方案

[Agent 6a: 成本分析师]     → { totalCost, breakdown }

[Agent 6b: 风险评估专家]   → { riskLevel, risks[] }

[Agent 6c: 碳排放评估专家] → { totalEmission, rating(A-E) }
          三路结果汇聚 (串行执行, 逐步展示 AI 思考过程)

[Agent 7a: 综合决策专家]
  输入: costData + riskData + carbonData
  输出: { recommendation: "break|keep", confidence(0-100), reasoning, keyFactors }
```

```
[Agent 7b: 最终报告生成器]
  输入: productPlan + competitorAnalysis + selectedSupplier + selectedProcess + analy
  输出: { businessBase, supplyChain, productionTimeline, lifecycle }
```

注：第 6 步三路评估采用串行执行（非 Promise.all），目的是在前端逐步展示每个 Agent 的实时思考过程，保证用户体验的连贯性。

### 4.3.3 SSE 流式输出实现

流式版本（/stream/route.ts）使用 ReadableStream + TextEncoder 实现 Server-Sent Events：

```
const stream = new ReadableStream({
  async start(controller) {
    const encoder = new TextEncoder();

    // 1. 发送开始信号
    controller.enqueue(encoder.encode(`data: ${JSON.stringify({ type
   : 'start' })}\n\n`));

    let fullContent = '';

    // 2. 调用 DeepSeek streamChat, 逐 token 推送
    await deepseek.streamChat(
      [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: userMessage }
      ],
      (chunk) => {
        const content = chunk.choices?.[0]?.delta?.content || '';
        if (content) {
          fullContent += content;
          controller.enqueue(encoder.encode(
            `data: ${JSON.stringify({ type: 'content', content })}\n
   \n`
          ));
        }
      },
      { temperature: 0.8, max_tokens: 3000 }
```

```
    );

    // 3. JSON 修复后发送结构化数据
    const parsed = tryParseWithFallback(fullContent);
    if (parsed) {
      controller.enqueue(encoder.encode(
        `data: ${JSON.stringify({ type: 'json', data: parsed })}\n\n
   `
      ));
    }

    // 4. 发送结束信号
    controller.enqueue(encoder.encode(`data: ${JSON.stringify({ type
   : 'end' })}\n\n`));
  }
});

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive'
  }
});
```

deepseek.streamChat（lib/deepseek.ts）直接调用 DeepSeek API 的 stream: true 接口，通过 ReadableStream reader 逐行解析 SSE 数据帧，回调 onChunk 处理每个 delta：

```
async streamChat(messages, onChunk, options = {}) {
  const response = await fetch(`${this.baseURL}/chat/completions`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${this.apiKey}`, 'Content-
   Type': 'application/json' },
    body: JSON.stringify({
      model: options.model || 'deepseek-chat',
      messages,
      temperature: options.temperature || 0.7,
      max_tokens: options.max_tokens || 2000,
      stream: true
```

```
    }),
    signal: AbortSignal.timeout(120000)
  });

  const reader = response.body?.getReader();
  const decoder = new TextDecoder();
  let buffer = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    const lines = buffer.split('\n');
    buffer = lines.pop() || '';
    for (const line of lines) {
      const trimmed = line.trim();
      if (trimmed.startsWith('data: ')) {
        const data = trimmed.slice(6);
        if (data === '[DONE]') return;
        try { onChunk(JSON.parse(data)); } catch {}
      }
    }
  }
}
```

前端事件格式：

- `{ type: 'start' }` —流开始
- `{ type: 'content', content: '...' }` —实时 token 片段
- `{ type: 'json', data: {...} }` —修复后的结构化 JSON
- `{ type: 'error', message: '...' }` —错误信息
- `{ type: 'end' }` —流结束

### 4.3.4 JSON 容错修复策略

模型输出的 JSON 偶有格式偏差，`stream/route.ts` 实现了四层修复策略：

```
const tryParse = (str: string): any | null => {
  try { return JSON.parse(str); } catch { return null; }
};

const braceCount = (str: string) =>
  (str.match(/{/g) || []).length - (str.match(/}/g) || []).length;

function tryParseWithFallback(raw: string): any | null {
  // 第1层：直接解析
  let parsed = tryParse(raw);
  if (parsed) return parsed;

  // 第2层：移除尾部逗号
  const fixed1 = raw.replace(/,\s*}/g, '}').replace(/,\s*]/g, ']');
  parsed = tryParse(fixed1);
  if (parsed) return parsed;

  // 第3层：提取最外层完整 JSON 对象
  const start = raw.indexOf('{');
  const end = raw.lastIndexOf('}');
  if (start !== -1 && end !== -1) {
    const extracted = raw.slice(start, end + 1);
    parsed = tryParse(extracted.replace(/,\s*}/g, '}').replace(/,\s
   *]/g, ']'));
    if (parsed) return parsed;
  }

  // 第4层：括号平衡补全
  const balance = braceCount(raw);
  if (balance > 0) {
    parsed = tryParse(raw + '}'.repeat(balance));
    if (parsed) return parsed;
  }

  return null;
}
```

### 4.3.5 进度持久化与断点续分析

每个零件的分析进度独立存储在 ProductionAnalysisProgress 表，以 (sessionId, partId) 为唯一索引：

```
model ProductionAnalysisProgress {
  id                      String   @id @default(cuid())
  sessionId               String
  partId                  String
  partName                String
  currentStep             String   @default("product-planning")
  productPlan             Json?
  competitorAnalysis      Json?
  suppliers               Json?
  selectedSupplier        Json?
  customizationQuestions  Json?
  customizationAnswers    Json?
  processes               Json?
  selectedProcess         Json?
  analysisResult          Json?
  finalReport             Json?
  isCompleted             Boolean  @default(false)
  createdAt               DateTime @default(now())
  updatedAt               DateTime @updatedAt
  @@unique([sessionId, partId])
}
```

API 层（/api/production-analysis-progress/route.ts）：

- **GET**：按 sessionId + partId 查询，恢复 currentStep 和已完成的各步结果
- **POST**：upsert 操作，存在则 update，不存在则 create，每步完成后立即写入
- **DELETE**：重置进度，支持用户重新分析

用户中途离开后重新进入，前端读取 currentStep 直接跳转到对应步骤，已完成的 Agent 结果从数据库恢复，无需重新调用 API。

### 4.3.6 模板预热与缓存命中

对于 32 个预设模板，7 份 Agent 报告已提前生成并存入 TemplateSession.agentReports 字段（Json 类型）。

lib/template-interceptor.ts 实现两级匹配引擎，内存索引 TTL 为 5 分钟：

```
// Level 1 精确匹配 (confidence = 1.0)
if (normalized.includes(nk) || nk.includes(normalized)) {
  return { template, confidence: 1.0 };
}

// Level 2 模糊匹配 (confidence = 0.8)
// Levenshtein 编辑距离 <= 2 且长度相似度 >= 0.6
const dist = levenshtein(normalized, nk);
const similarity = Math.min(normalized.length, nk.length) / Math.max
   (normalized.length, nk.length);
if (dist <= 2 && similarity >= 0.6) {
  return { template, confidence: 0.8 };
}
```

命中模板时，agentReports 直接从数据库读取返回，整个 Agent 链路响应时间从 30-60 秒降至 $<$ 100ms，同时保留真实 AI 路径供非模板物品使用。

### 4.3.7 与传统单次 AI 调用的对比

| 维度 | 传统单次调用 | 多 Agent 协同 |
|---|---|---|
| Prompt 复杂度 | 极高，难以维护 | 每个 Agent Prompt 简洁聚焦 |
| 输出质量 | 多任务权衡，质量参差 | 专业领域深度输出 |
| 可扩展性 | 修改一处影响全局 | 独立 Agent 可单独迭代 |
| 延迟 | 单次长调用，全程等待 | 流式输出 + 并行执行 |
| 可观测性 | 黑盒，难以调试 | 每步输出可独立检查 |
| 容错性 | 单点失败全链路崩溃 | 单 Agent 失败可降级处理 |

## 4.4 前端技术栈详解

### React Flow 画布引擎

拆解树使用 React Flow 11 渲染，节点布局由 Dagre 算法自动计算（有向图分层布局）。每个节点是自定义的 MatterNode 组件，支持：

- 展示 emoji 图标 + Wikimedia 配图（懒加载）
- 点击触发 AI 拆解（展开子节点）
- 点击已展开节点弹出组件详情面板
- 节点位置拖拽后自动保存至 Session

节点数据结构（types/graph.ts）：

```
interface MatterNode {
  id: string;            // UUID
  label: string;         // 显示名称
  type: 'root' | 'component' | 'raw_material';
  parentId: string | null;
  dimension?: 'mechanical' | 'electronic' | 'chemical' | 'process';
  isExpanded: boolean;   // 是否已拆解
  isTerminal: boolean;   // 是否为原材料终止节点
  icon?: string;         // emoji
  imageUrl?: string;     // Wikimedia 图片 URL
  searchTerm?: string;   // 图片搜索词
}
```

四种内容维度（ContentDimension）为节点提供语义标注，支持按维度过滤视图：

- mechanical：纯机械结构（齿轮、轴承、壳体）
- electronic：电子元件（芯片、传感器、PCB）
- chemical：化学配方（材料成分、涂层、电解质）
- process：加工工艺（冲压、焊接、热处理）

### Framer Motion 动画系统

首页视差动画完全由 useScroll + useTransform 驱动，将 scrollYProgress（0-1）映射为各层的位移、缩放、透明度、模糊值：

```
// 视频层：随滚动模糊并淡出
const videoBlur = useTransform(scrollYProgress, [0, 0.25], [0, 15]);
const videoOpacity = useTransform(scrollYProgress, [0, 0.4, 0.6],
   [1, 0.6, 0]);
```

```
// 立方体三面依次拆解
const faceTopY = useTransform(scrollYProgress, [0, 0.02, 0.08,
   0.12], [0, 0, -50, -50]);
const faceLeftX = useTransform(scrollYProgress, [0, 0.08, 0.14,
   0.18], [0, 0, -50, -50]);
const faceRightX = useTransform(scrollYProgress, [0, 0.14, 0.20,
   0.24], [0, 0, 50, 50]);
```

整个动画区域高度为 1000vh，固定定位层叠，纯 CSS transform 驱动，无 JS 动画帧，GPU 加速流畅。

## 4.5 数据库设计要点

**SQLite + Prisma 的选型理由：**

- 开发阶段零配置，文件即数据库（prisma/dev.db）
- Prisma 6 支持 Json 字段类型，拆解树、Agent 报告等复杂结构直接存储为 JSON，无需额外表设计
- 生产环境可无缝切换至 PostgreSQL(Neon/Vercel Postgres)，只需修改 DATABASE_URL

**关键索引设计：**

```
// 用户会话按更新时间和最后访问时间双索引，支持历史列表排序
@@index([userId, updatedAt])
@@index([userId, lastAccessedAt])

// 生产分析进度唯一约束，防止重复创建
@@unique([sessionId, partId])

// 模板按 key、分类、维度三维索引，支持快速检索
@@index([templateKey])
@@index([category])
@@index([dimensions])
```

**自动入库机制（instrumentation.ts）：**
Next.js 15 的 instrumentationHook 在服务器启动时触发 register() 函数，自动检测数据库中是否有模板，若为空则扫描 prisma/seeds/templates/ 目录下的 32 个 JSON 文件，逐一校验（字段完整性、关键词格式、树结构、7 份报告数量）后批量 upsert 入库。这确保了任何新环境部署后无需手动执行 seed 命令。

## 4.6 认证与安全

- **NextAuth.js v4**，Credentials Provider，bcryptjs 密码哈希
- **JWT 策略**，30 天有效期，token 中携带 `user.id`
- **Middleware 保护**：`/deconstruct`、`/history`、`/api/deconstruct`、`/api/identify`、`/api/knowledge-card`、`/api/sessions/*` 均需认证
- **注意**：当前 middleware 的 `authorized` 回调返回 `true`（即只要有 token 即可），实际鉴权在各 API Route 内部通过 `getServerSession` 二次验证

## 4.7 工具链与工程配置

| 工具 | 版本 | 用途 |
|---|---|---|
| Next.js | 15.1.3 | 全栈框架，App Router |
| React | 19.0.0 | UI 框架 |
| TypeScript | 5.7.2 | 类型安全 |
| Prisma | 6.19.2 | ORM + 迁移 |
| NextAuth | 4.24.13 | 认证 |
| React Flow | 11.11.4 | 图谱可视化 |
| Dagre | 0.8.5 | 图布局算法 |
| Framer Motion | 11.15.0 | 动画 |
| Tailwind CSS | 3.4.17 | 样式 |
| Playwright | 1.58.2 | E2E 测试（含 MCP 调试） |
| tsx | 4.21.0 | TypeScript 脚本执行 |
| Docker | — | 容器化部署（含 Dockerfile） |

**Next.js 配置优化点（`next.config.js`）：**

- `serverActions.bodySizeLimit: '10mb'`：支持大图片上传
- `instrumentationHook: true`：启用服务器启动钩子
- `workerThreads: false, cpus: 1`：限制编译并发，降低开发机内存压力

- `onDemandEntries`：优化开发服务器页面缓存策略
- `images.remotePatterns`：允许加载 Wikimedia 图片

---

# 五、可访问入口

| 入口类型 | 地址 |
|---|---|
| 本地开发 | `http://localhost:3000`（`npm run dev`） |
| 生产构建 | `npm run build && npm start` |
| Docker 部署 | `docker build -t break-it-down . && docker run -p 3000:3000 break-it-down` |
| 创空间/演示链接 | *(待补充)* |

**快速启动：**

```
# 1. 安装依赖
npm install

# 2. 配置环境变量
cp .env.local.example .env.local
# 填写 DASHSCOPE_API_KEY 或 AI_BASE_URL+AI_API_KEY
# 填写 NEXTAUTH_SECRET (openssl rand -base64 32)
# 填写 DATABASE_URL (默认 file:./prisma/dev.db)

# 3. 初始化数据库
npx prisma migrate dev

# 4. 启动 (模板自动入库)
npm run dev
```

---

# 六、项目亮点总结

1. **多 Agent 协同**是本项目最核心的技术创新——7 个专业 Agent 串行协作 + 第 6 步三路并行，将一个复杂的制造业决策流程完整数字化，每个 Agent 职责单一、

上下文链式传递、SSE 实时流式输出，兼顾了分析深度、响应速度和用户体验。

2. **模板拦截 + 内存索引**实现了 AI 调用与预计算结果的无缝切换，在演示场景下提供毫秒级响应，在真实场景下保留完整 AI 能力，是工程实用主义的典型体现。

3. **递归拆解 + 强制终止规则**确保了拆解树的语义完整性——任何物品最终都会收敛到 12 种基础原材料，形成一个有意义的知识闭环，而非无限递归的技术展示。

4. **全链路持久化**（会话、节点位置、分析缓存、生产分析进度）让用户的每一次探索都有价值，支持随时中断、随时恢复，降低了使用门槛。

5. **沉浸式视差首页**将产品理念（拆解、定制）融入交互设计，立方体三面拆解动画直接呼应了产品核心功能，品牌一致性极强。

Break It Down 拆解的是物理的外壳，重构的是未来的供应链。在定制时代，让每个人都拥有定制自己产品的能力。

**Break It Down → Build Infinite Dreams!**